

Construction Management Field App

RAYNS Technologies

Technical Discussion & Way forward



Objectives

- **Digitize field operations** for issue logging, RFI management, and document handling.
- **Enhance collaboration** between site teams, project managers, QA/QC inspectors, and stakeholders.
- **Ensure offline functionality** with seamless background synchronization when connectivity is restored.
- **Provide secure role-based access** to project data and operations.
- **Lay the foundation for advanced visualization** tools (3D models, photogrammetry) in future releases.

Key Functional Modules

Frontend

Authentication & Access Control

- Secure login (OAuth2 or equivalent) with **biometric authentication** (Fingerprint / Face ID).
- Role-based dashboards
- Permissions matrix for **create, read, update, delete (CRUD)** operations.

Issue & RFI Management

- Log, edit, assign, and track issues with media attachments, location, and comments.
- Status workflow: *Open → In Progress → Resolved → Closed*.
- RFI submission & responses.
- Offline creation with background synchronization.

UI/UX

- Threaded Comments for RFI
- Ease of use Thinking
- Push Notifications

Key Functional Modules

Backend

Document Management

- Hierarchical folder structure: *Project > Discipline > Document Type > Files*.
- Upload, download, and view all file types; view PDFs, CSVs, TXT, and ASCII-based files in-app.
- Built-in viewer with zoom, search, page navigation.

Offline Support

- Offline-first architecture with local caching.
- Manual and auto-cleanup options with space monitoring.
- Background sync with conflict resolution and sync logs.

Visualization & Future Readiness

- Support for WebView-based integration of Autodesk Forge, Unity WebGL, and photogrammetry tools.
- Lazy loading for large assets.
- Modular integration of visualization tools
- GPU Acceleration

Non-Functional Capabilities

- Performance:** App launch in under 2 seconds; smooth navigation.
- Reliability:** Offline-first design with retry mechanisms.
- Scalability:** Support for thousands of documents/issues.
- Security:** Encrypted local storage, HTTPS APIs, token-based auth.
- Usability:** Minimal taps for key actions, responsive UI for varied screen sizes.
- Maintainability:** Modular codebase, configurable sync/cleanup rules.

Technical Approach

1. Architecture

1. **Cross-platform development** using **Flutter/React Native** for shared codebase across Android & iOS.
2. **Offline-first architecture** using local databases (Firestore/SQLite/Room/Realm).
3. Cloud-hosted APIs with secure HTTPS endpoints and token authentication.

2. Data Synchronization

1. Background sync scheduler with retry and conflict resolution strategies.

3. Security Implementation

1. OAuth2.0 authentication with biometric support.

4. Future Integrations

1. WebView container for advanced model viewing without impacting initial release timeline.

Development Approach

Feature / Area	Approach A: Flutter + Firebase	Approach B: React Native + Custom Backend
Frontend	Flutter (single codebase for Android/iOS)	React Native (single codebase for Android/iOS)
Backend	Firebase Firestore, Firebase Storage, Firebase Cloud Functions	Custom REST API (FastAPI/Node.js) + SQL/NoSQL DB
Hosting	Firebase Hosting	Self-hosted server or cloud VM (AWS/DO/Heroku)
Auth	Firebase Auth	Custom JWT-based authentication
Push Notifications	Firebase Cloud Messaging	OneSignal / Firebase Cloud Messaging
Offline Support	Built-in Firestore offline sync	Custom offline cache implementation
Development Speed	Faster initial development	Slightly slower, more setup
Scalability	Easy for small-to-mid scale apps	Highly scalable for complex apps
Cost	Low infra cost, pay-as-you-go	Hosting + backend maintenance cost
Maintenance	Minimal backend maintenance	Requires backend/server maintenance

Advantages & Considerations

Approach A: Flutter + Firebase

- ✓ Fast to market
- ✓ Lower maintenance cost
- ✓ Built-in real-time sync & offline mode
- ⚠ Vendor lock-in to Google Firebase pricing & ecosystem

Approach B: React Native + Custom Backend

- ✓ Full control over data & architecture
- ✓ Easier migration & integration with 3rd-party services
- ✓ Optimized offline support for large data sets
- ⚠ Higher initial development effort & ongoing server maintenance

Project Plan & Timeline

Phase	Flutter + Firebase (Beginner-Friendly)	React Native + Custom Backend
Requirements Finalization	1–2 weeks	1–2 weeks
UI/UX Design	4–5 weeks	4–5 weeks
Core Feature Development	6–8 weeks	10–14 weeks
Backend Setup & API Development	N/A (Firebase)	5–7 weeks
Integration & Testing	3–4 weeks	4–5 weeks
UAT (User Acceptance Testing) & Feedback	2 weeks	3 weeks
Deployment & Launch	2 week	2 week
Total	~16–20 weeks	~26–34 weeks

Deliverables

- Fully functional Android & iOS app meeting agreed specifications.
- API documentation and admin configuration manual.
- User training material (PDF + video tutorials).
- Source code repository access.
- Post-deployment support for 3 months.

Assumptions and Queries

- ❑ **User Base: Estimate Number of user required for architecture finalization.**
- ❑ **Access Functionalities**
 - Site Engineer: Log issues with photos, assign to someone, add location
 - Project Manager: See all issues/RFIs, assign work, change status, approve
 - QA/QC Inspector: Check quality issues, update status, add comments
 - Client/Stakeholder: Read-only view of progress and documents
 - Admin: Manage users, roles, and settings
- ❑ **Built-in viewer with zoom, search, page navigation, and print options. (Development Heavy Feature suggested alternative download and view)**

Risks & Mitigation

- **Performance on low-end devices** → Optimize image compression, caching may be slow.
- **API limitations (if using free image providers)** → Might have to Preload image sets offline.
- **Firebase pricing jumps with scale** → will need Monitoring usage, optimizing queries.

Thank You

RAYNS Technologies

